Project - High Level Design

on

Autonomous Logistics Researcher Agent

Course Name: Agentic AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1	HATIM BOHRA	EN22CS301418
2	INDRA KAMAL PATEL	EN22CS301434
3	GOUTAM JAT	EN22CS301371
4	ISHIKA AGRAWAL	EN22CS301442
5	HARSH SOLANKI	EN22CS301394
6	HITESH AHUJA	EN22CS301426

Group Name: Group 09D4

Project Number: AAI – 45

Industry Mentor Name:

University Mentor Name:

Academic Year: 2025-26

Table of Contents

1. **Introduction.**
   1.1. **Scope of the document.**
   This Low Level Design (LLD) document provides granular technical specifications for every software component of the Autonomous Logistics Researcher Agent. It covers internal module design, agent configuration, task prompts, tool integration, data flow between components, and persistence strategy — at a level of detail sufficient for a developer to implement the system from scratch without additional design artefacts.
   This document does not address deployment infrastructure, CI/CD pipelines, or business requirements (covered in HLD and BRD).

   1.2. **Intended Audience**
   - Backend developers implementing agent modules in Python 3.10+
   - ML engineers configuring LLM prompts and CrewAI orchestration
   - QA engineers designing test cases for self-correction behaviour

   1.3. **System overview.**
   The Autonomous Logistics Researcher Agent is a multi-agent AI system that automates deep-dive market research in the global logistics and supply chain domain. Given a complex research query, the system autonomously navigates the live web, extracts and cross-verifies data points from authoritative sources (BIMCO, Freightos, Lloyd's List, port authority bulletins), and synthesises findings into a structured, schema-compliant Markdown report persisted to a local knowledge repository for future RAG use.

   Core capabilities: Autonomous web search · multi-source verification · self-correction on thin results (max_iter=5) · structured Markdown output · RAG-ready knowledge persistence via ChromaDB.

2. **System Design.**
   2.1. Application Design
   The application is structured as a layered Python package where each layer has a single responsibility and communicates through well-defined interfaces. The diagram below shows the complete system architect

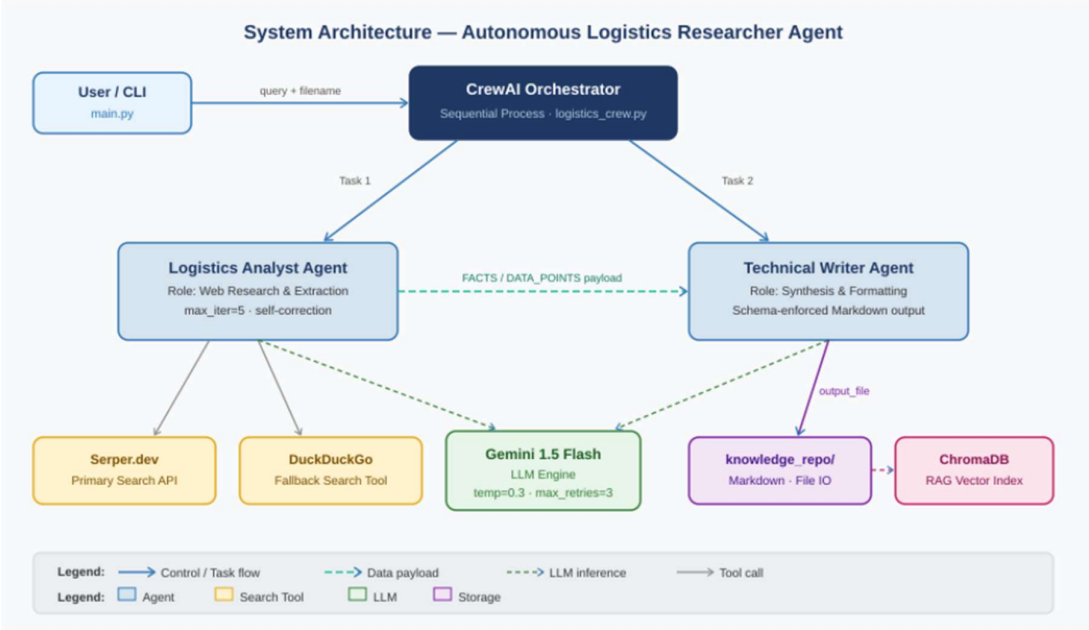


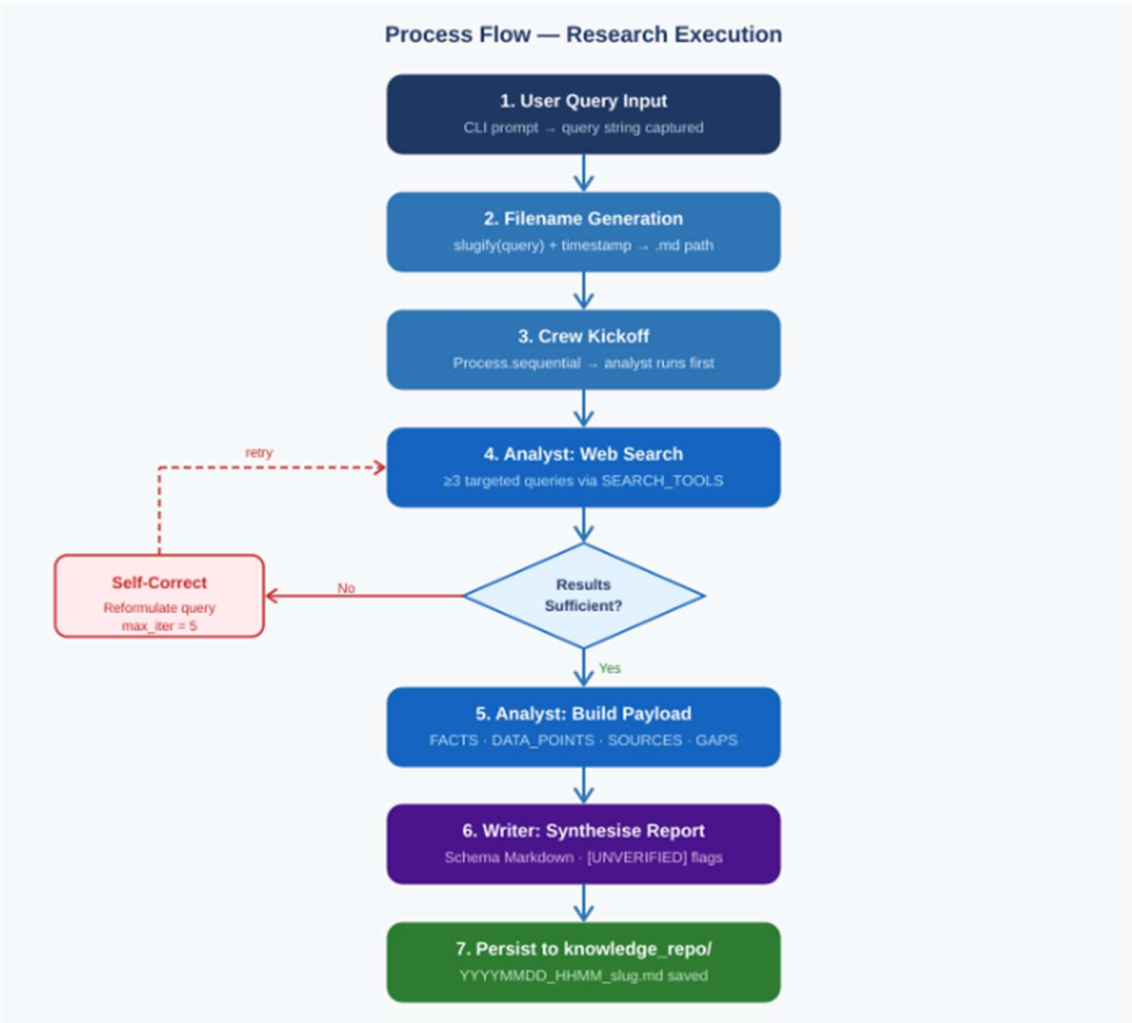
Figure 1 — System Architecture Diagram

2.2. Process Flow.

The diagram below illustrates the complete execution flow from user query input to persisted Markdown report, including the self-correction loop triggered when search results are insufficient.

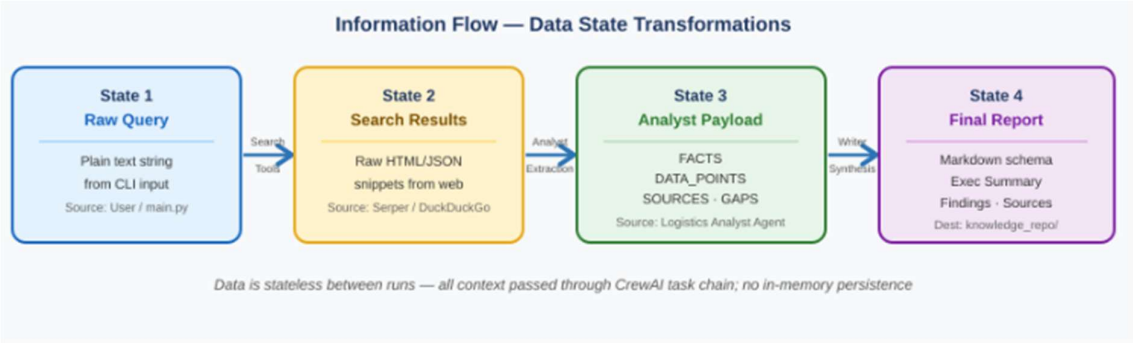
Step-by-Step Process

Step	Actor	Action
1	User / main.py	User enters a logistics research query via CLI prompt
2	main.py	slugify(query) + datetime.now() generates unique output filename
3	logistics_crew.py	Crew assembled with Process.sequential; crew.kickoff() invoked
4	Logistics Analyst	Runs ≥3 targeted web searches via SEARCH_TOOLS; extracts facts, metrics, URLs
5	Logistics Analyst	Self-correction: if results thin/stale, reformulates query and retries (max_iter=5)
6	CrewAI Orchestrator	Analyst output passed as context to Technical Writer task
7	Technical Writer	Synthesises FACTS/DATA_POINTS/SOURCES payload into Markdown report
8	CrewAI / File IO	Report saved to knowledge_repo/{filename}.md via output_file parameter



2.3. Information Flow.

Data undergoes four distinct state transformations as it moves through the system. No state is persisted in memory between runs — all context is carried in the CrewAI task chain and written to disk at completion.



2.4. Components Design

The component diagram below shows the internal structure of each module and its dependencies.

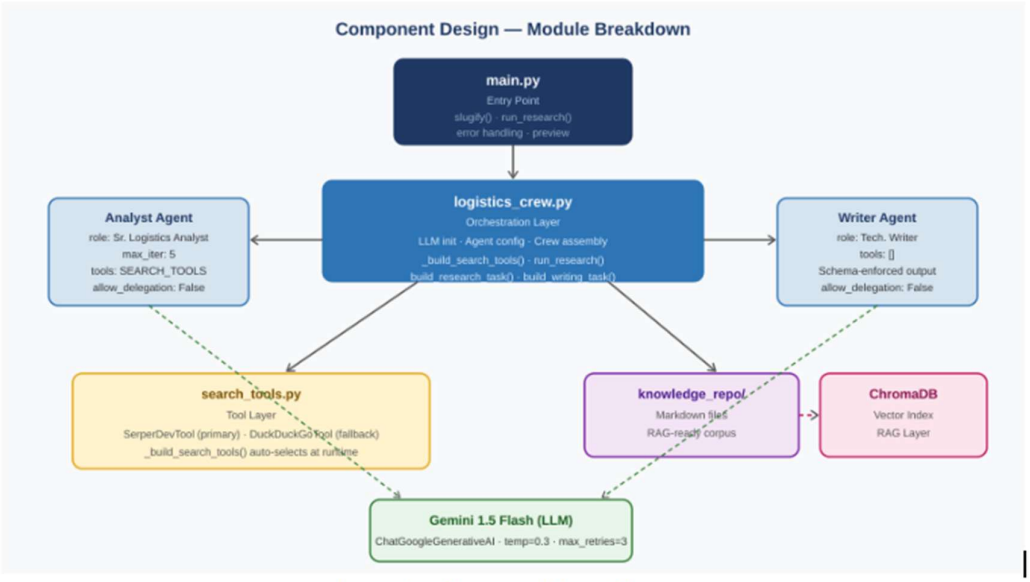


Figure 4 — Component Design Diagram

2.5. Key Design Considerations

Consideration	Design Decision & Rationale
Self-Correction	max_iter=5 on the Analyst. CrewAI re-invokes with previous output as context when LLM evaluates result as insufficient.
Tool Modularity	SEARCH_TOOLS assembled at startup by _build_search_tools(). Swapping to a premium API requires changing one return statement.
Prompt Isolation	All prompt text lives in Task description strings, not agent backstory. Agents are reusable; prompt tuning is localised.
Statelessness	No in-memory state between runs. All context carried via CrewAI task chain. Enables horizontal scalability.
API Key Safety	Keys loaded from .env via python-dotenv. Hard fail at import time if GOOGLE_API_KEY missing — no silent errors.
LLM Swappability	Single llm object injected into both agents. Switching to GPT-4o or Claude requires changing only the constructor.
Rate Limit	max_retries=3 + request_timeout=60 handle HTTP 429 / 503 with

Handling	LangChain exponential backoff decorator.
RAG Extensibility	knowledge_repo/ naming, Markdown schema, and section headers chosen to maximise chunk quality for MarkdownTextSplitter.

2.6 API Catalogue

All external APIs and internal Python interfaces consumed or exposed by the system are documented below.

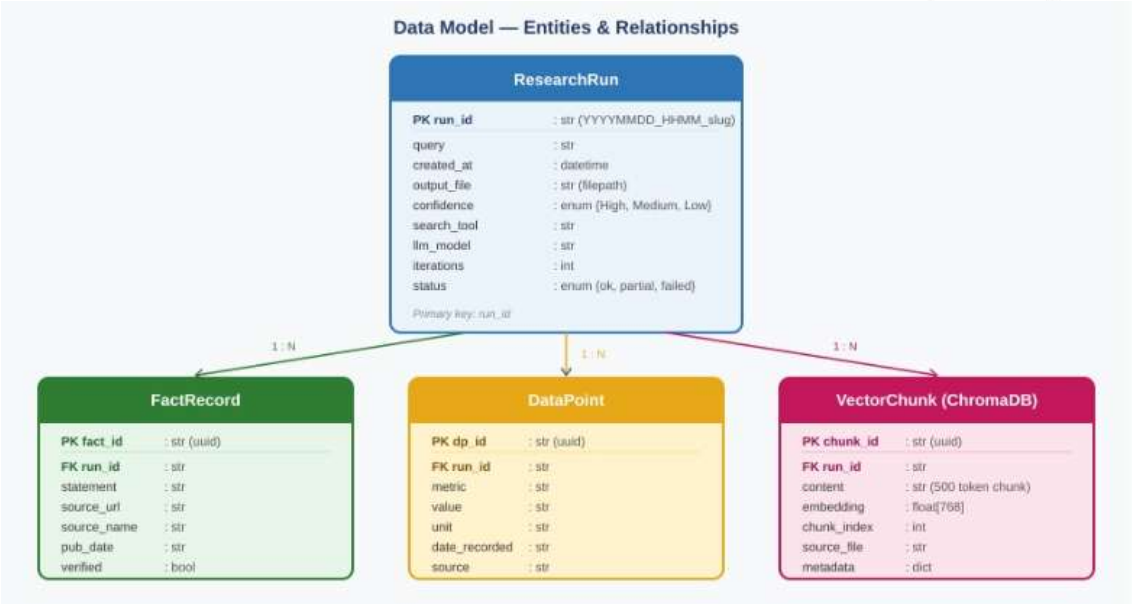
External APIs

API	Endpoint	Auth	Used For
Google Gemini	generativelanguage.googleapis.com	GOOGLE_API_KEY	LLM inference for both agents
Serper.dev	google.serper.dev/search	SERPER_API_KEY	Primary web search (optional)
DuckDuckGo	duckduckgo.com (scrape)	None	Free fallback web search
Gemini Embeddings	models/embedding-001	GOOGLE_API_KEY	RAG vector indexing (Phase 5)

3. Data Design.

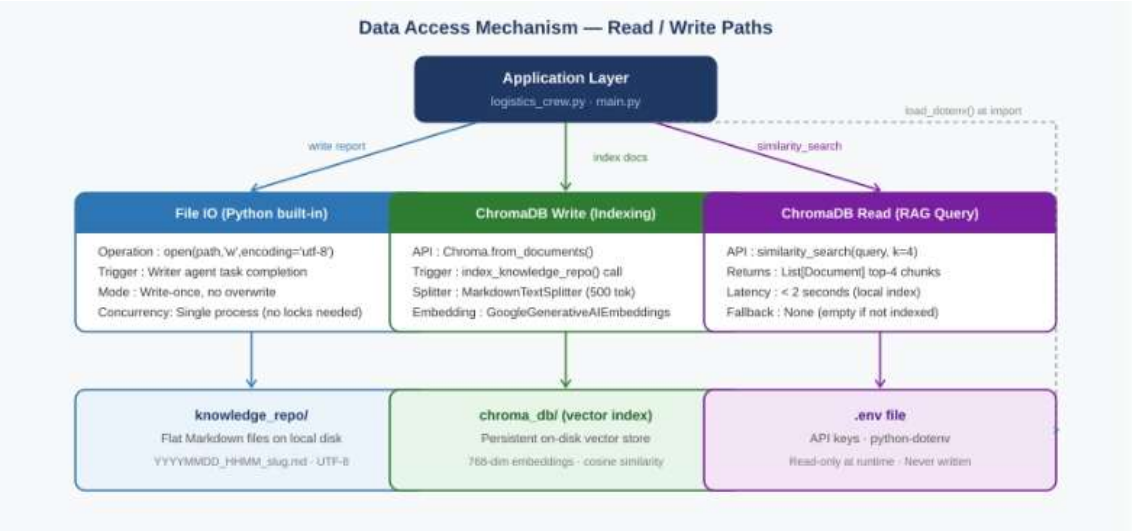
3.1. Data Model

The system uses a flat-file data model with no relational database. All persistent data lives in knowledge_repo/ as plain Markdown files. Logically, the model contains four entities: ResearchRun (one per execution, holds query metadata and run details), FactRecord (individual verified or flagged statements extracted during research), DataPoint (quantitative metrics such as freight rates and transit times), and VectorChunk (ChromaDB embeddings of chunked report text used for semantic search). Each FactRecord, DataPoint, and VectorChunk carries a foreign key back to its parent ResearchRun via the run_id field.



3.2. Data Access Mechanism

The system has three data access paths. The File IO path uses Python's built-in open() to write the completed Markdown report once at the end of each run — write-only, no locking needed. The ChromaDB write path is triggered manually via index_knowledge_repo(), which reads all .md files, splits them into 500-token chunks, embeds them using embedding-001, and persists the vector collection to chroma_db/. The ChromaDB read path is used for RAG queries via similarity_search(question, k=4), returning the top-four semantically similar chunks in under two seconds. The .env file is read once at module import by python-dotenv and is never written to at runtime.



3.3. Data Retention Policies

The knowledge _repo/ directory is append-only — files are never modified or deleted by the system, and there is no TTL or expiry. Every past report retains long-term value as a RAG corpus entry. The chroma_db/ directory is a derived artefact and can be deleted and fully rebuilt from knowledge_repo/ at

any time using index_knowledge_repo(). The .env file is static and managed only by the developer. In-memory run state is ephemeral — it exists only for the duration of one run_research() call and is garbage collected on completion.

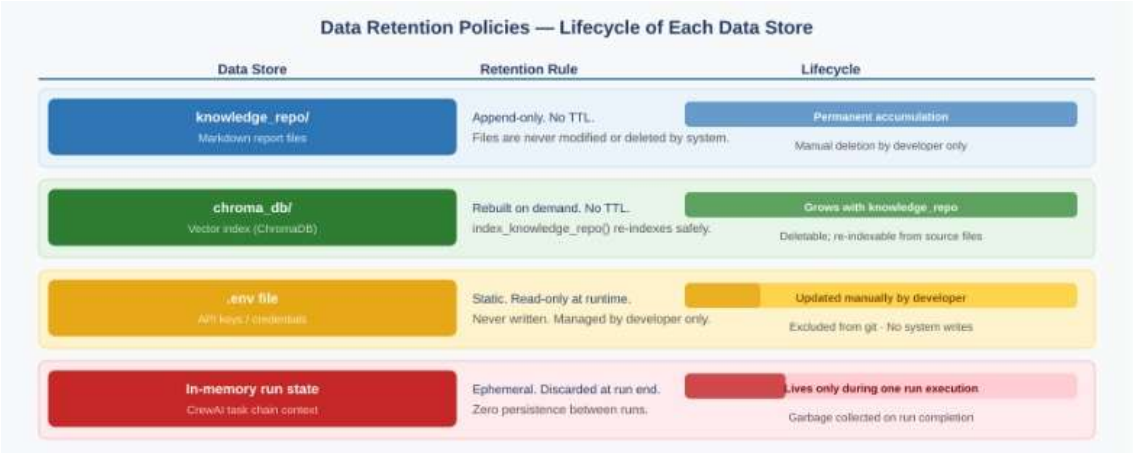


Figure 4 — Data Retention Policies: Lifecycle per Data Store

3.4. Data Migration

Three migration scenarios are relevant to this system. Schema evolution — when a new report section is added — requires only updating the Task description prompt; old files remain valid because the schema is additive by design. ChromaDB rebuild — needed if the index is corrupted or the embedding model changes — involves deleting chroma_db/ and re-running index_knowledge_repo() against the unchanged knowledge_repo/ files. Directory relocation — moving the project to a new path or server — requires copying both directories, updating the output_file path and glob in indexer.py, and re-running the indexer. In all three cases, no data is lost and no downtime is required.

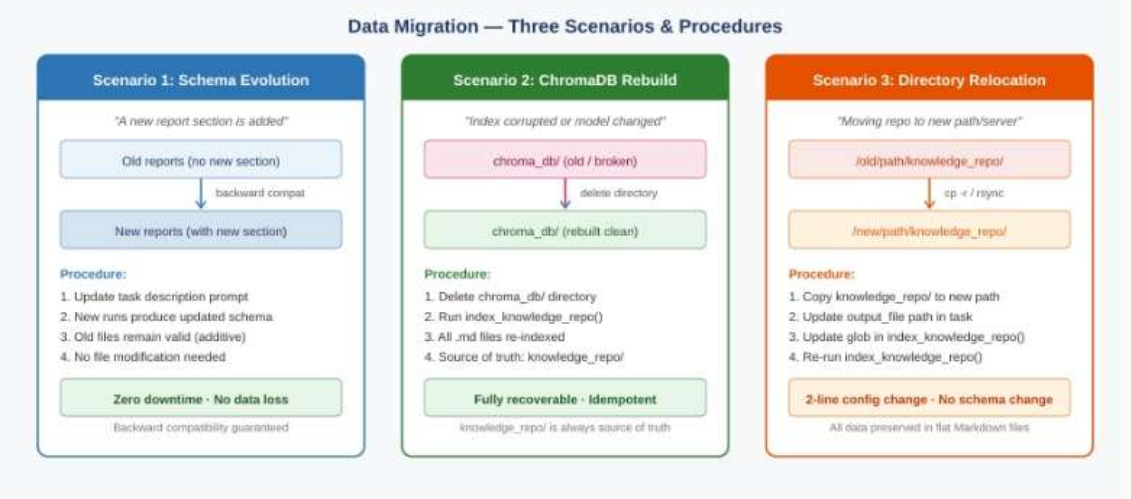


Figure 5 — Data Migration: Three Scenarios & Procedures

4. Interfaces

The system exposes no REST API or web server. It communicates through three interface types: external HTTPS API calls to Serper.dev, DuckDuckGo, and Gemini; internal Python function calls between modules; and CLI/file I/O for input and output.

Figure 6 — System Interfaces & State Management

5. State and Session Management

The system is fully stateless. There is no session, no user account, no authentication token, and no shared memory between runs. Each call to run_research() is completely independent. Within a single run, state is carried only through the CrewAI task chain object, which is garbage collected when the function returns. The run transitions through four states: IDLE → RESEARCHING → WRITING → PERSISTED.

State	Trigger	Description
IDLE	System start	Waiting at CLI prompt. No active query.
RESEARCHING	crew.kickoff()	Analyst searching the web. Self-correction loop may run.
WRITING	Analyst task complete	Writer synthesising and formatting the Markdown report.
PERSISTED	Writer task complete	Report saved to disk. System returns to IDLE.

6. Caching

The system uses a two-level caching strategy. Level 1 is the file cache — knowledge_repo/ accumulates Markdown reports that can be reused via the RAG layer, avoiding redundant API calls for similar queries. Level 2 is the vector cache — ChromaDB persists a semantic index of all past reports on disk, enabling similarity_search(question, k=4) to return relevant past context in under two seconds. There is no runtime cache (no Redis or Memcached) because the system is stateless and single-process. LLM responses and live search results are intentionally not cached since logistics data changes frequently and freshness is a core requirement.

7. Non-Functional Requirements

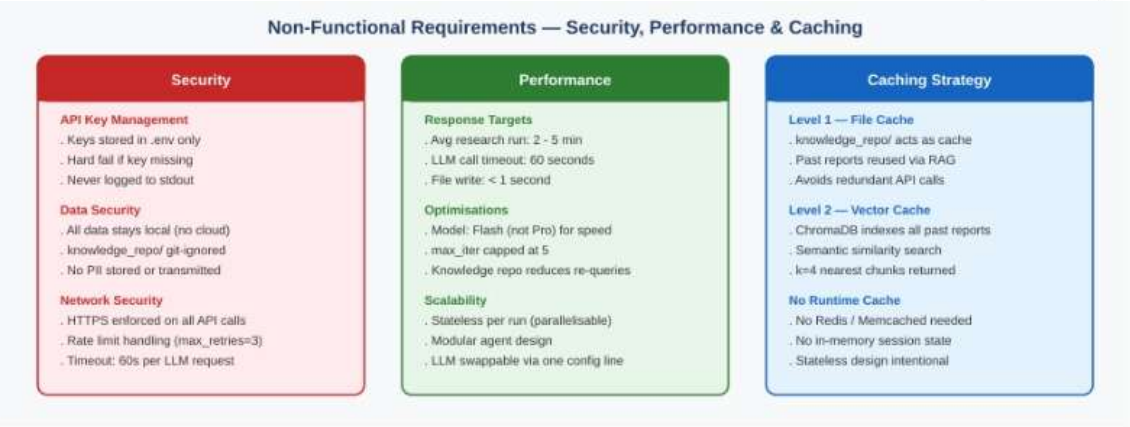


Figure 7 — Security, Performance & Caching Overview

7.1 Security Aspects

Area	Requirement	Implementation
API Key Management	Never hardcoded or logged	.env via python-dotenv; hard fail if GOOGLE_API_KEY missing
Data at Rest	No sensitive data in version control	knowledge_repo/ and chroma_db/ in .gitignore; no PII stored
Data in Transit	All external calls encrypted	HTTPS enforced on all API calls
Rate Limiting	Graceful throttling handling	max_retries=3 with backoff; request_timeout=60s
Input Safety	No prompt injection	Query passed as plain string; not eval'd; temp=0.3

7.2 Performance Aspects

Metric	Target	Design Decision
End-to-end run	2 – 5 minutes	Gemini Flash; max_iter capped; RAG reduces re-queries
LLM call latency	< 60 sec / call	request_timeout=60; max_retries=3 for transient errors
File write	< 1 second	Plain file IO; UTF-8 Markdown; no DB overhead
RAG query	< 2 seconds	ChromaDB in-process; k=4; local embedding, no network
Memory	< 512 MB RAM	Stateless per run; ChromaDB lazy-loads index segments

3. References

#	Resource	URL / Citation
1	CrewAI Documentation	https://docs.crewai.com
2	LangChain Google GenAI	https://python.langchain.com/docs/integrations/llms/google_ai
3	Google Gemini API Docs	https://ai.google.dev/docs
4	Serper.dev API	https://serper.dev/docs
5	DuckDuckGo Search (PyPI)	https://pypi.org/project/duckduckgo-search
6	ChromaDB Documentation	https://docs.trychroma.com
7	LangChain Chroma Integration	https://python.langchain.com/docs/integrations/vectorstores/chroma
8	python-dotenv	https://pypi.org/project/python-dotenv
9	CrewAI BaseTool Reference	https://docs.crewai.com/concepts/tools
10	Gemini Embeddings Guide	https://ai.google.dev/docs/embeddings_guide